

# Homework #1

## Data Structure

Yu-Ting Wu

# Homework Description

- **Goals**
  - Implement **run-length encoding** and **decoding** for **text** and **image** data
- **Grading**
  - **Part I: text**
    - Correctness of text encoding 45%
    - Correctness of text decoding 30%
  - **Part II: image**
    - Correctness of image encoding 10%
    - Correctness of image decoding 10%
  - **A small report (to explain your implementation)** 5%

# Run-Length Encoding

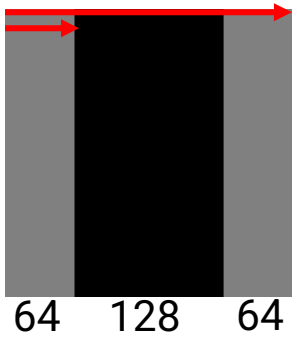
- **Run-length encoding (RLE)** is a widely-used technique for **lossless** data compression
- Runs of data (sequences in which the same data value occurs in many consecutive data elements) are stored as a **single data value** and **count**

E.g., text data:

wwwwaaadexxxxxx ➡ w4a3d1e1x6

E.g., image data:

| color | pixel value |
|---|---|
| | 0 |
| | 128 |
| | 255 |




64 128 128 (64+64)

128 128 … 128 0 0 … 0 128 128 … 128 ..

➡ 128 64 0 128 128 128 ..

# Run-Length Decoding

- **Run-length decoding** takes a run-length-encoded string and decompresses it into its original data

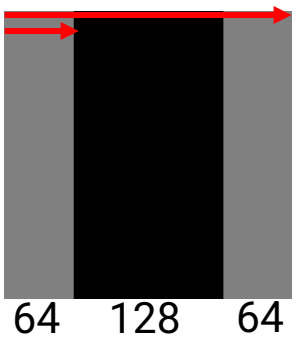
E.g., text data:

w4a3d1e1x6 ⇒ wwwwaaadexxxxxx

E.g., image data:

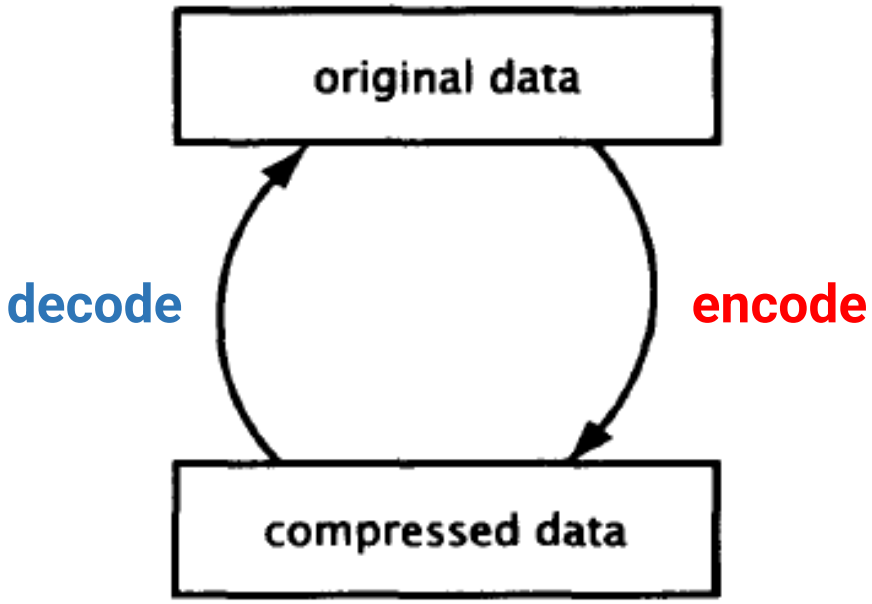
128 64 0 128 128 128 ⇒ 128 128 … 128 0 0 … 0 128 128 … 128 …

64 128 128



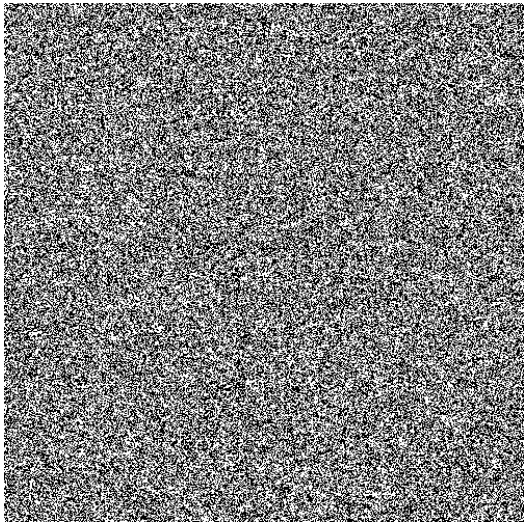
# Run-Length Coding (cont.)

- Run-length encoding is **lossless**



# Run-Length Coding (cont.)

- For data with many consecutive same values, **RLE** is very efficient
  - E.g., simple graphics images such as icons or line drawing
- For files that do not have many runs, **RLE** could greatly **increase** the file size
  - E.g., a random image



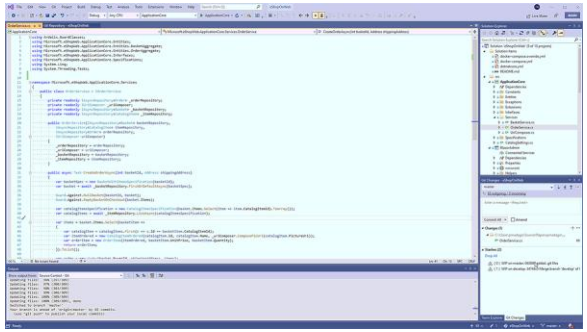
# Skeleton Code

- Please download the **skeleton code** from 數位學苑3.0
  - As mentioned in the first class, we will use **Microsoft Visual Studio Community 2022/2026** for the IDE (open with ***.sln / *.slnx**)
  - You can easily find tutorials on the Internet (need to install C++ feature)









- All the libraries you need are located in the "**OpenCV**" folder and have been configured in the project

# Skeleton Code (cont.)

- Please download the **skeleton code** from 數位學苑3.0
- All the libraries you need are located in the "**OpenCV**" folder and have been configured in the project
- **Files**

**DON'T NEED TO TOUCH**

  - **HW1.cpp:** program entry point, calling and validating each component of HW1
  - **Data.h:** define the data structure of encoding/decoding data
  - **ImageUtil.h:** provide utilities functions for processing images

  - **RunLengthCoding.h:**
  - **RunLengthCoding.cpp:**

C++ class for implementing run-length encoding and decoding (**TODO**)

# Skeleton Code (cont.)

- **HW1.cpp**

```
int main(int argc, char** argv)
{
    // Part1: text encoding/decoding.
    RunTextCodingCheck();

    cout << endl;
    cout << "========================================================================" << endl;
    cout << endl;

    // Part2: image encoding/decoding.
    RunImageCodingCheck();

    return 0;
}
```

**we will first test the text inputs then the image inputs**

# Skeleton Code (cont.)

- **HW1.cpp**

```cpp
// Test text inputs.
const int nTestTexts = 5;
const char* testTexts[nTestTexts] = {
    "wwwwaaadexxxxxx",
    "aaaaaaaaaaaaaaaaaaabbbbbbbbbcccccccccccccccccccccccccccccccddd",
    ">>>>>>>>>>>>000000<<<<<<<<<<<<<<<<",
    "!!!!!!!!!!!^^^^^^^^<<<<<<<WWWWWWWWWWWWWWWW",
    "pneumonoultramicroscopicsilicovolcanoconiosis"
};
```

```cpp
void RunTextCodingCheck()
{
    for (int i = 0; i < nTestTexts; ++i) {
        cout << "Check text input " << i << endl;
        cout << "[INPUT] original test text input: " << testTexts[i] << endl;

        // Prepare for the test text.
        const char* testText = testTexts[i];
        Text origText(testText);

        // Perform encoding.
        TextEncodeData textEncodeData;
        RunLengthCoding::EncodeText(origText, textEncodeData);

        // Perform decoding.
        Text decodeText;
        RunLengthCoding::DecodeText(textEncodeData, decodeText);

        // Validation of text encoding/decoding.
        if (ValidateTextCoding(origText, decodeText))
            cout << "[RESULT] test text input " << i << " passed!" << endl;
        else
            cout << "[RESULT] test text input " << i << " failed..." << endl;

        cout << "------------------------------------------------------------" << endl;
        Sleep(waitTime);
    }
}
```

get the test text input

```cpp
char* data;
int length;
```

Text

call your encoder to encode text

call your decoder to decode text

check if the input text string equals the output decoded one

# Skeleton Code (cont.)

- **Data.h**

```cpp
// Struct Text definition.
struct Text
{
    Text()
    {
        data = nullptr;
        length = 0;
    }
    Text(const char* str)
    {
        length = strlen(str);
        data = new char[length+1];
        strncpy_s(data, length+1, str, length+1);
    }
    ~Text()
    {
        if (data ≠ nullptr) {
            delete [] data;
            data = nullptr;
        }
    }
    char* data;
    int length;
};
```

```cpp
// Struct TextEncodeData definition.
struct TextEncodeData
{
    TextEncodeData()
    {
        data = nullptr;
        dataLength = 0;
    }
    ~TextEncodeData()
    {
        if (data ≠ nullptr) {
            delete [] data;
            data = nullptr;
        }
    }
    char* data;          // The encoded char data stream.
    int dataLength;      // The length of the encoded char data stream.
};
```

**(We don't have numbers in input text! Thus, when you see a number, you know it means count)**

data = "wwwwaaadexxxxxx"
length = 15



data = "w4a3d1e1x6"
dataLength = 10

# Skeleton Code (cont.)

- **HW1.cpp**

```cpp
// Test image inputs.
const int nTestImages = 5;
const char* testImageFiles[nTestImages] = {
    "TestInputs/Simple.png",
    "TestInputs/SatoruGojo.png",
    "TestInputs/AttackOnTitan.png",
    "TestInputs/SpyFamily.png",
    "TestInputs/DemonSlayer.png"
};
```

```cpp
void RunImageCodingCheck()
{
    for (int i = 0; i < nTestImages; ++i) {
        cout << "Check image input " << i << ": " << endl;

        // Load the test image.
        cv::Mat origImg = cv::imread(testImageFiles[i], cv::IMREAD_GRAYSCALE);
        ImageUtil::ShowImage(origImg, "Image before comprssion (downsampled)");

        // Perform encoding.
        ImageEncodeData imgEncodeData;
        RunLengthCoding::EncodeImage(origImg, imgEncodeData);

        // Perform decoding.
        cv::Mat decodeImg;
        RunLengthCoding::DecodeImage(imgEncodeData, decodeImg);
        ImageUtil::ShowImage(decodeImg, "Image after decomprssion (downsampled)");

        // Validation of image encoding/decoding.
        if (ImageUtil::SameUCharImage(origImg, decodeImg))
            cout << "[RESULT] test image input " << i << " passed!" << endl;
        else
            cout << "[RESULT] test image input " << i << " failed..." << endl;

        cout << "------------------------------------------------------------" << endl;
        Sleep(waitTime);
    }
}
```

load the test image input

call your encoder to encode image

call your decoder to decode image

check if the input image equals the output decoded one

# Skeleton Code (cont.)

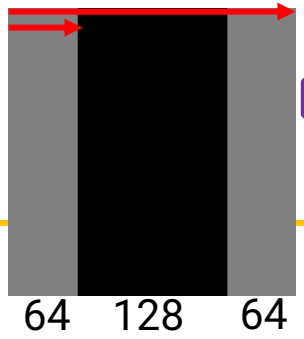
- **Data.h**

```
// Struct ImageEncodeData Declarations.
struct ImageEncodeData
{
    ImageEncodeData()
    {
        data = nullptr;
        numData = imageWidth = imageHeight = imageType = 0;
    }
    ~ImageEncodeData()
    {
        if (data ≠ nullptr) {
            delete [] data;
            data = nullptr;
        }
    }
    ImageRunLengthData* data;    // Encoded ImageRunLengthData data array.
    int numData;                 // The number of items in the encoded data array.

    // Data for reconstructing the original image.
    int imageWidth;      // Original image width.
    int imageHeight;     // Original image height.
    int imageType;       // Original image type.
};
```

```
// Image coding data.
// ---------------------------------------------------------------------
// Struct ImageRunLengthData Declarations.
struct ImageRunLengthData
{
    ImageRunLengthData()
    {
        value = 0;
        count = 0;
    }
    ImageRunLengthData(uchar val, int c)
    {
        value = val;
        count = c;
    }
    uchar value;         // Pixel value.
    int count;           // The successive occurrence of the pixel value.
};
```

**uchar: unsigned char (0~255)**




**count**
128 64 0 128 128 128 ...
**value**

# Skeleton Code (cont.)

- **HW1.cpp**
  - Five test images can be found in the **“TestInputs”** folder



Simple.png



DemonSlayer.png




SatoruGojo.png




AttackOnTitan.png



SpyFamily.png

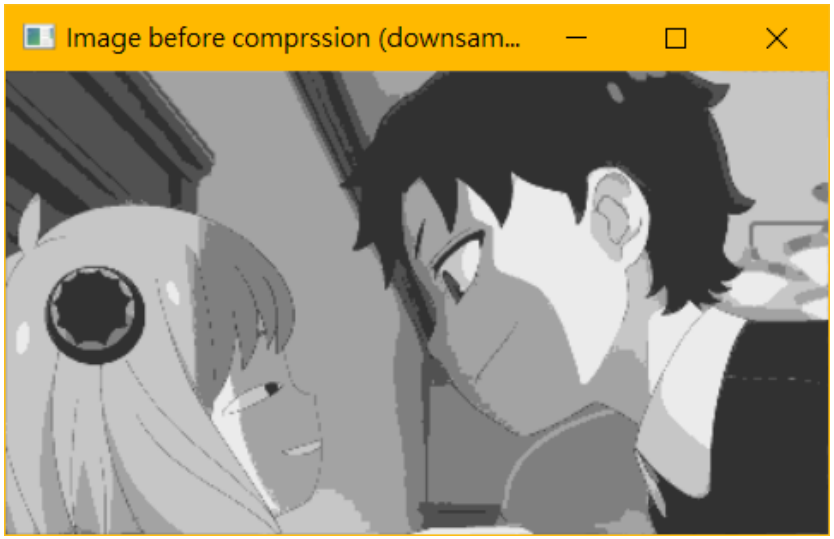
# Skeleton Code (cont.)

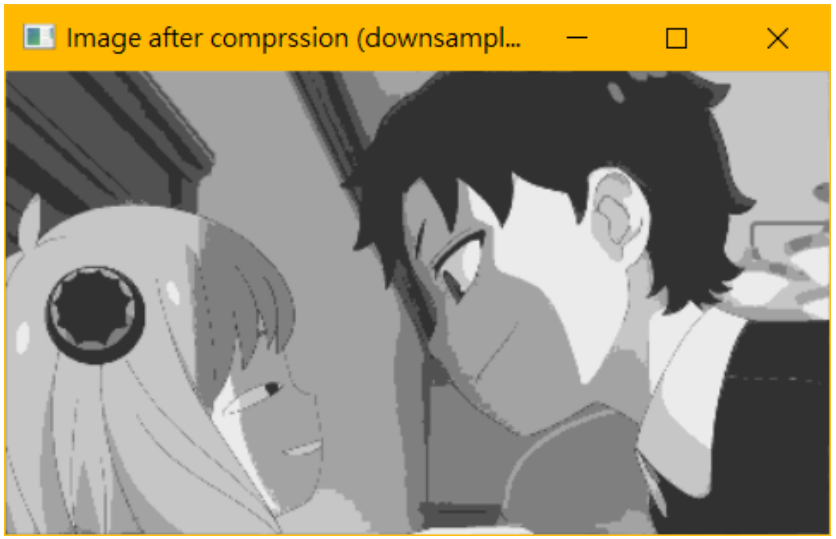
- **HW1.cpp**
  - You can remove the comment on

    **ImageUtil::ShowImage(origImg, "Image before comprssion (downsampled)");**

    **ImageUtil::ShowImage(decodeImg, "Image after comprssion (downsampled)");**

    to show the input / decoded image




**press “spacebar” on the window can proceed the program**

# Skeleton Code (cont.)

- **HW1.cpp**
  - If your program works well, you can see the command line showing a "**PASS**" message
  - Otherwise, you will get a "**FAIL**" message



# Skeleton Code (cont.)

- **RunLengthCoding.h**

```
// Class RunLengthCoding Declarations.
class RunLengthCoding
{
public:
    // RunLengthCompression Public Methods.
    RunLengthCoding() {};
    ~RunLengthCoding() {};

    // Text Encoding/Decoding Methods.
    static void EncodeText(const Text& origText, TextEncodeData& encodeData);
    static void DecodeText(const TextEncodeData& encodeData, Text& decodeText);

    // Image Encoding/Decoding Methods.
    static void EncodeImage(const cv::Mat &image, ImageEncodeData& encodeData);
    static void DecodeImage(const ImageEncodeData &encodeData, cv::Mat& decodeImage);
};
```

pass by reference !

# Pass by Reference

- Recall we have introduced **call by address (pointer)** for modifying local parameters
- C++ provide a more concise way to pass parameters named **call by reference**

```
void PlusOne(int* n)
{
    *n = *n + 1;
}

int main(int argc, char** argv)
{
    int v = 3;
    PlusOne(&v);
    cout << v << endl;   4

    return 0;
}
```

```
void PlusOne(int& n)
{
    n = n + 1;
}

int main(int argc, char** argv)
{
    int v = 3;
    PlusOne(v);
    cout << v << endl;   4

    return 0;
}
```

# Skeleton Code (cont.)

- **RunLengthCoding::EncodeText**

```
void RunLengthCoding::EncodeText(const Text& origText, TextEncodeData& encodeData)
{
    // Check if the input is valid.
    if (origText.data == nullptr || origText.length <= 0) {
        cerr << "Invalid text input for encoding..." << endl;
        exit(1);
    }
    cout << "Original length: " << origText.lengt

    // Implement the run-length text encoding algorithm here.
    // ---------------------------------------------------------------------------------
    // TODO: add code here to encode the origText and put your result in encodeData.
    // Note: you should fill both encodeData.data and encodeData.dataLength.
    // ---------------------------------------------------------------------------------

    // Print out encode information (PLEASE DO NOT TOUCH THIS CODE).
    if (encodeData.data != nullptr) {
        cout << "Encode string: " << encodeData.data << endl;
        cout << "Encode length: " << encodeData.dataLength << endl;
        cout << "Compression ratio: " << (float)origText.length / (float)encodeData.dataLength << endl;
    }
}
```

input text

output

```
char* data;         // The encoded char data stream.
int dataLength;     // The length of the encoded char data stream.
```

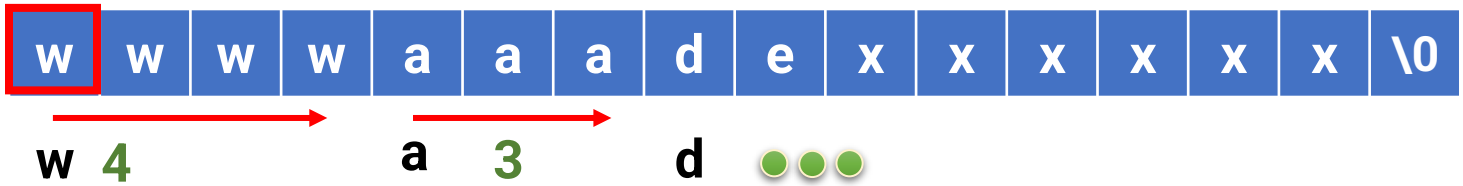
(fill your result in this struct)

Add your encoding code here!

# Skeleton Code (cont.)

- **RunLengthCoding::EncodeText**

wwwwaaadexxxxxx




w4a3d1e1x6

**encodeData.data**

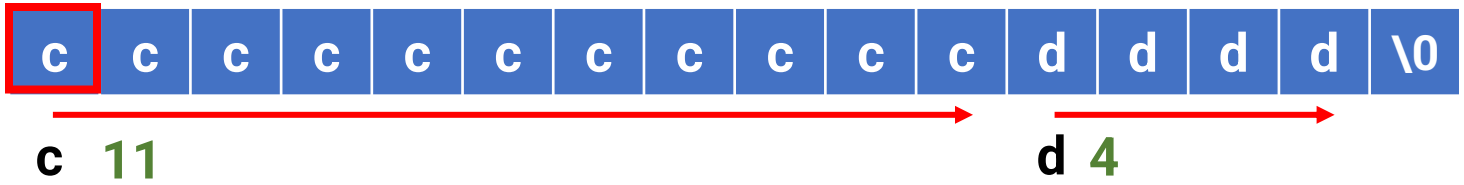



**encodeData.dataLength = 10**

# Skeleton Code (cont.)

- **RunLengthCoding::EncodeText**

ccccccccccdddd




c11d4

**encodeData.data**




**encodeData.dataLength = 5**

# Skeleton Code (cont.)

- **RunLengthCoding::DecodeText**

input encode data from your encoder

(fill your result in this struct)
output

```
void RunLengthCoding::DecodeText(const TextEncodeData& encodeData, Text& decodeText)
{
    // Implement the run-length text decoding algorithm here.
    // ------------------------------------------------------------------------------------
    // TODO: add your code here to decode the data from _encodeData_ and put the result into decodeText.
    // Note: you should fill both decodeText.data and decodeText.length.
    // ------------------------------------------------------------------------------------
    // Add your decoding code here!

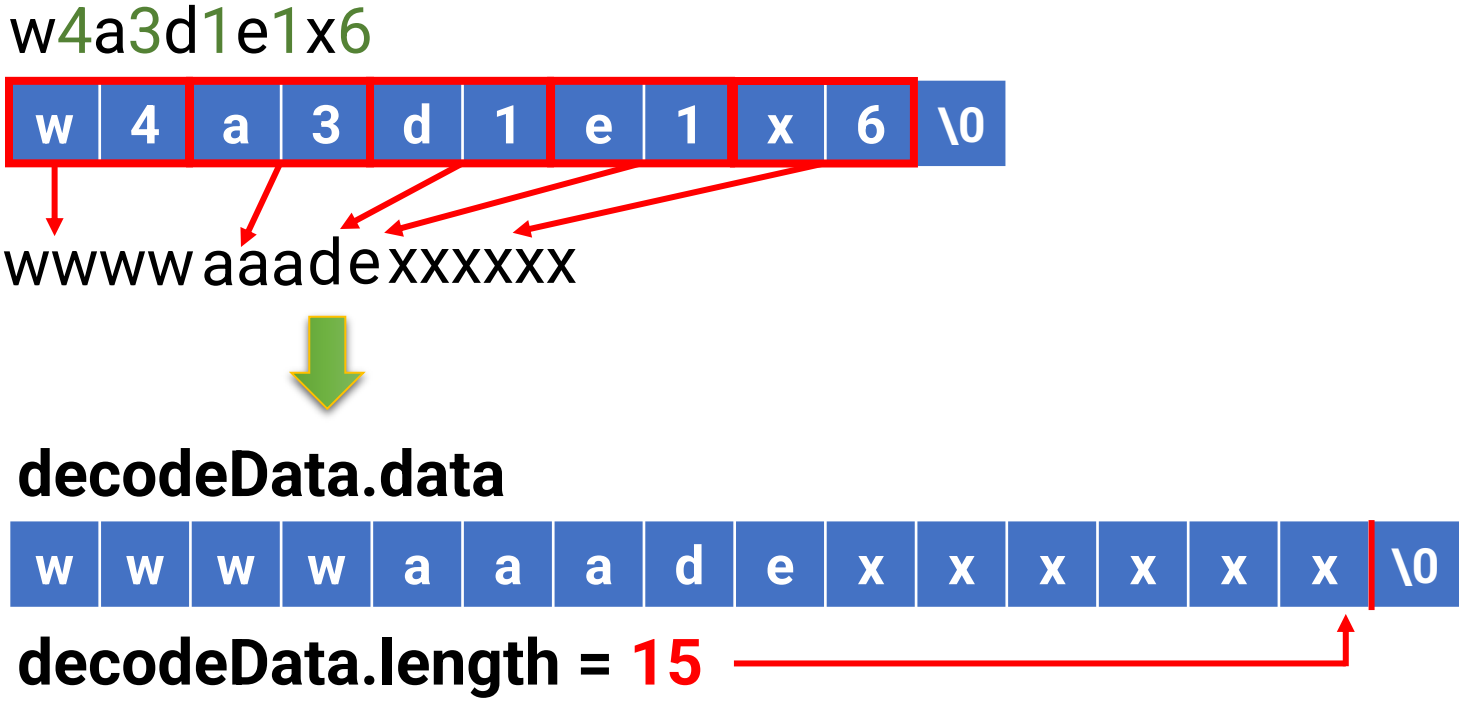
    // Print out decode result (PLEASE DO NOT TOUCH THIS CODE).
    if (decodeText.data ≠ nullptr) {
        cout << "Decode string: " << decodeText.data << endl;
    }
}
```

```
char* data;
int length;
```

Text

# Skeleton Code (cont.)

- **RunLengthCoding::DecodeText**




**(We don't have numbers in input text! Thus, when you see a number, you know it means count)**

# Skeleton Code (cont.)

- **RunLengthCoding::EncodeImage**

```
void RunLengthCoding::EncodeImage(const cv::Mat& image, ImageEncodeData& encodeData)
{
    // Keep image information (PLEASE DO NOT TOUCH THIS CODE).
    encodeData.imageWidth = image.size().width;
    encodeData.imageHeight = image.size().height;
    encodeData.imageType = image.type();

    // Obtain raw pixel data (PLEASE DO NOT TOUCH THIS CODE).
    uchar* rawImageData = image.data;
    int nPixels = encodeData.imageWidth * encodeData.imageHeight;
    int rawSize = sizeof(uchar) * nPixels;
    cout << "Raw image size: " << rawSize << " bytes" << endl;


    // Implement the run-length image encoding algorithm here.
    // ------------------------------------------------------------------------------------
    // TODO: add your code here to encode the image pixel data and put the result into encodeData.
    // Note: you should fill both encodeData.data and encodeData.numData.
    // ------------------------------------------------------------------------------------

    // Print out encode information (PLEASE DO NOT TOUCH THIS CODE).
    int encodedSize = sizeof(ImageRunLengthData) * encodeData.numData;
    cout << "Encoded size: " << encodedSize << " bytes" << endl;
    cout << "Compression ratio: " << (float)rawSize / (float)encodedSize << endl;
}
```

output
(fill your result in this struct)

```
ImageRunLengthData* data;
int numData;
```

```
uchar value;
int count;
```

the actual data you need to handle

Add your encoding code here!

# Skeleton Code (cont.)

- **RunLengthCoding::EncodeImage**

**uchar* rawImageData (uchar rawImageData[nPixels])**

| | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 128 | 128 | 128 | 128 | 0 | 0 | 0 | 0 | 128 | 128 | 128 | 128 |
| 128 | 128 | 128 | 128 | 0 | 0 | 0 | 0 | 128 | 128 | 128 | 128 |
| 128 | 128 | 128 | 128 | 0 | 0 | 0 | 0 | 128 | 128 | 128 | 128 |
| 128 | 128 | 128 | 128 | 0 | 0 | 0 | 0 | 128 | 128 | 128 | 128 |



value count

{128, 4} {0, 4} {128, 8} ●●●

**Fill in encodeData.data and encodeData.numData**

# Skeleton Code (cont.)

- **RunLengthCoding::DecodeImage**

input encoded data from your encoder

```
void RunLengthCoding::DecodeImage(const ImageEncodeData& encodeData, cv::Mat& decodeImage)
{
    // Implement the run-length image decoding algorithm here.

    int nPixels = encodeData.imageWidth * encodeData.imageHeight;
    uchar* decodeData = new uchar[nPixels];
    fill(decodeData, decodeData + nPixels, (uchar)0);
    // ------------------------------------------------------------------------------------
    // TODO: add your code here to decode the data from encodeData and fill into the decodeData array.
    // ------------------------------------------------------------------------------------

    // Fill the data in decodeData into OpenCV Mat (PLEASE DO NOT TOUCH THIS CODE).
    decodeImage = cv::Mat(cv::Size(encodeData.imageWidth, encodeData.imageHeight), encodeData.imageType);
    for (int h = 0; h < decodeImage.size().height ; ++h) {
        for (int w = 0; w < decodeImage.size().width; ++w) {
            decodeImage.at<uchar>(h, w) = decodeData[h * decodeImage.size().width + w];
        }
    }
}
```

fill this array (row-major), I will do the rest job for you

Add your decoding code here!

# Skeleton Code (cont.)

- **RunLengthCoding::DecodeImage**

{128, 4} {0, 4} {128, 8} ●●●

value count



uchar* decodeData (row-major)

| | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 128 | 128 | 128 | 128 | 0 | 0 | 0 | 0 | 128 | 128 | 128 | 128 |
| 128 | 128 | 128 | 128 | | | | | | | | |
| | | | | | | | | | | | |
| | | | | | | | | | | | |

# About the Test Cases

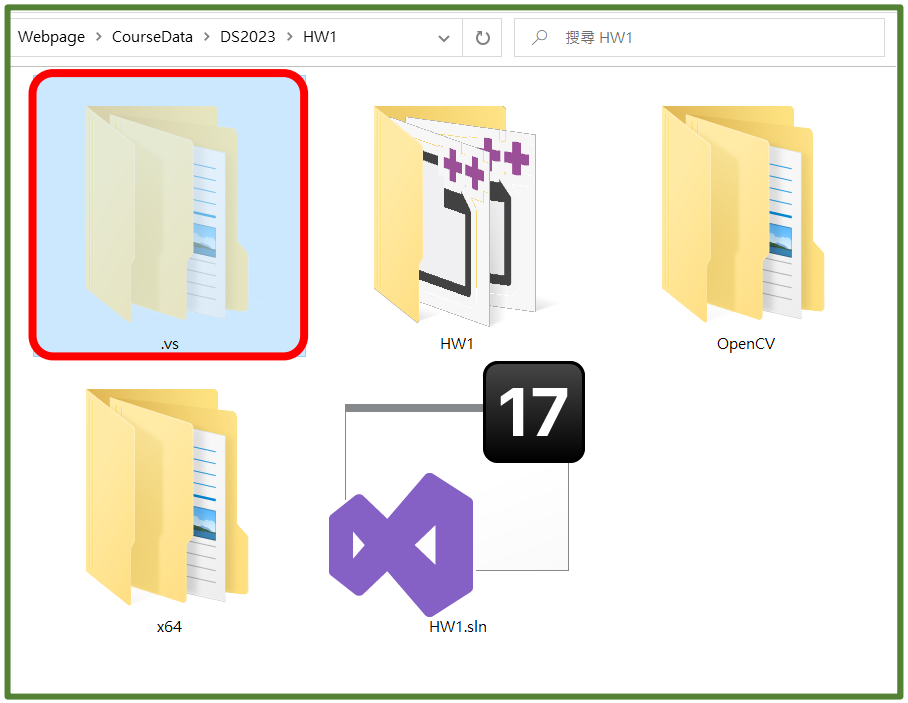
- For run-length **text** coding
  - To avoid ambiguities, the **text inputs** will **NOT** contain any **numbers (0~9)**
  - The **dataLength** in the **TextEncodeData** and the **length** in **Text** should **NOT** include the **null character for termination**
    - For example: the length of **w4a3d1e1x6** should be **10**
- TAs will validate your program with more test cases
- You can finish your HW with any C++ standard library
  - **But need to follow the interface of the methods**
- If you have questions, please sent **ME** an email

# Submission

- **Deadline: Oct. 20, 2026 (PM 11:59)**
- **Submission rule**
  - Please zip your **VisualStudio Community project** and the **report** to a file **named "學號_姓名_HW1.zip"**
  - Upload the zip file to "數位學苑3.0"
- **Late Policy**
  - One day 90%
  - Two days 80%
  - Three days 70%
  - Four days 60%
  - **> Four days NOT ACCEPTED**

# Submission (cont.)

- To **reduce the file size**, you can delete the **hidden** folder: **.vs** in your project




END OF PRESENTATION
ANY QUESTIONS?